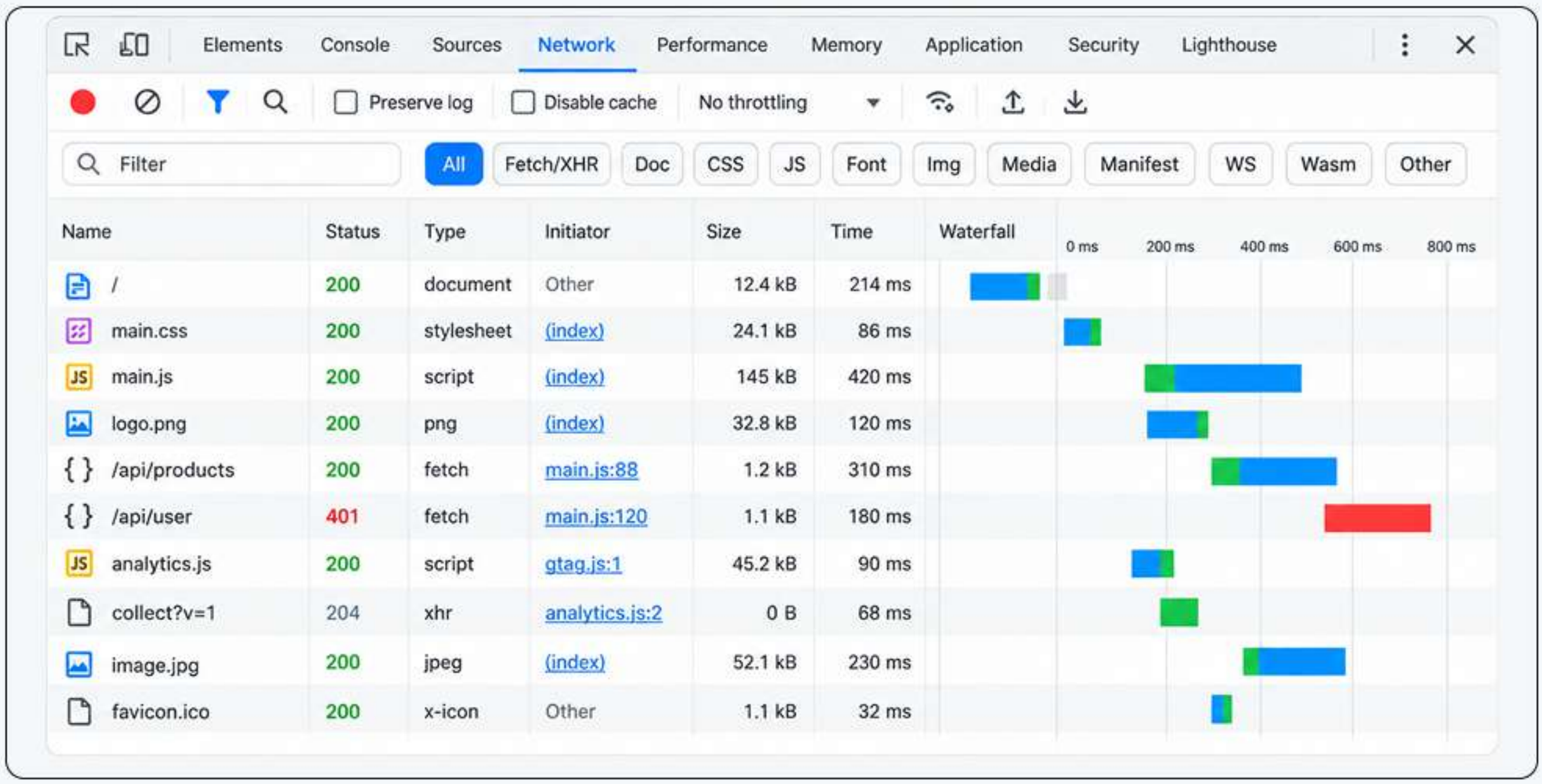
HOW TO READ THE NETWORK TAB

Understand What's Happening Between Your Browser And The Server.

CHROME DEVTOOLS EDITION

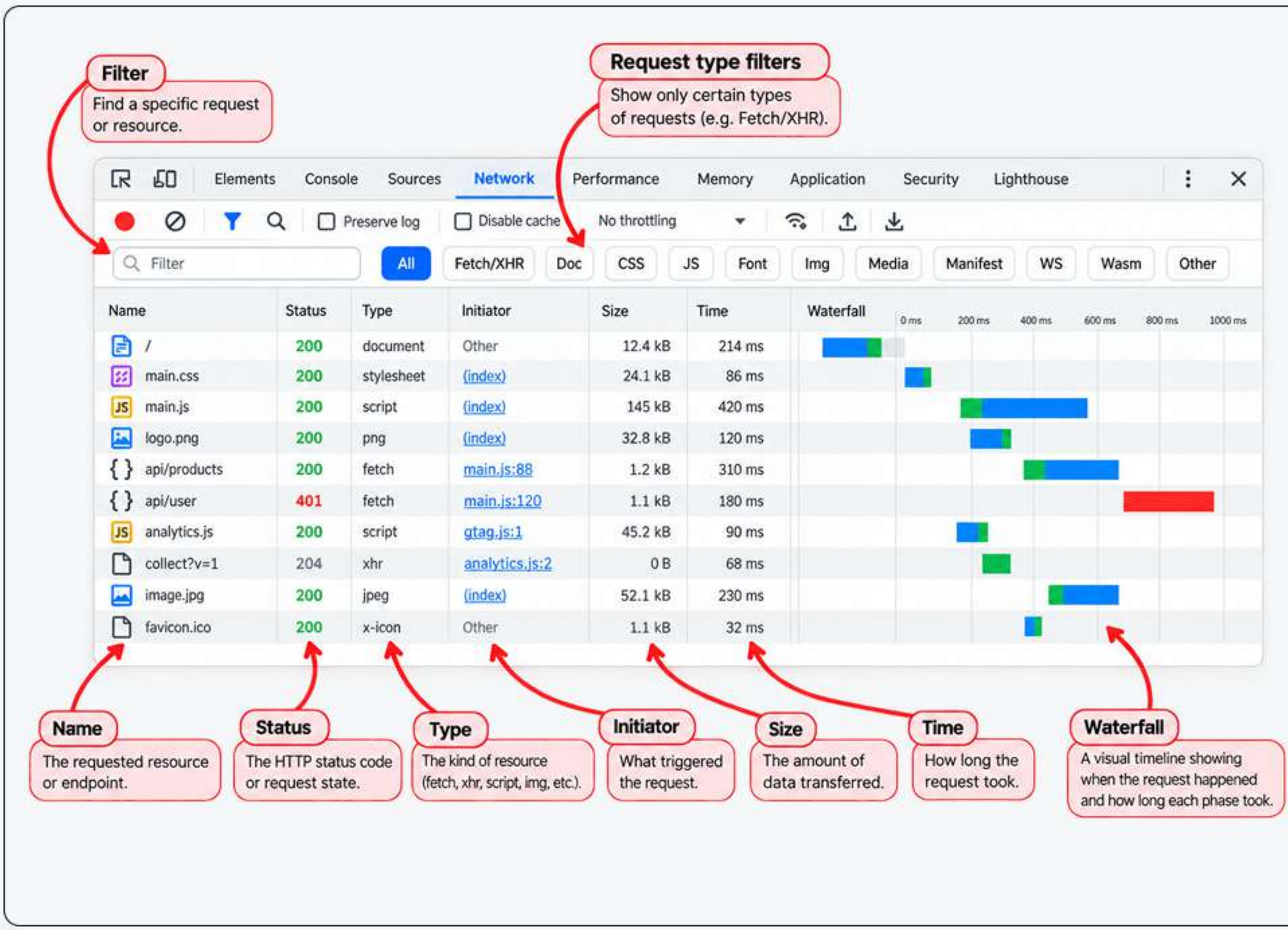


@this.girl.tech

What Is the Network Tab?

The Network tab shows the requests your browser makes while loading and using a website.

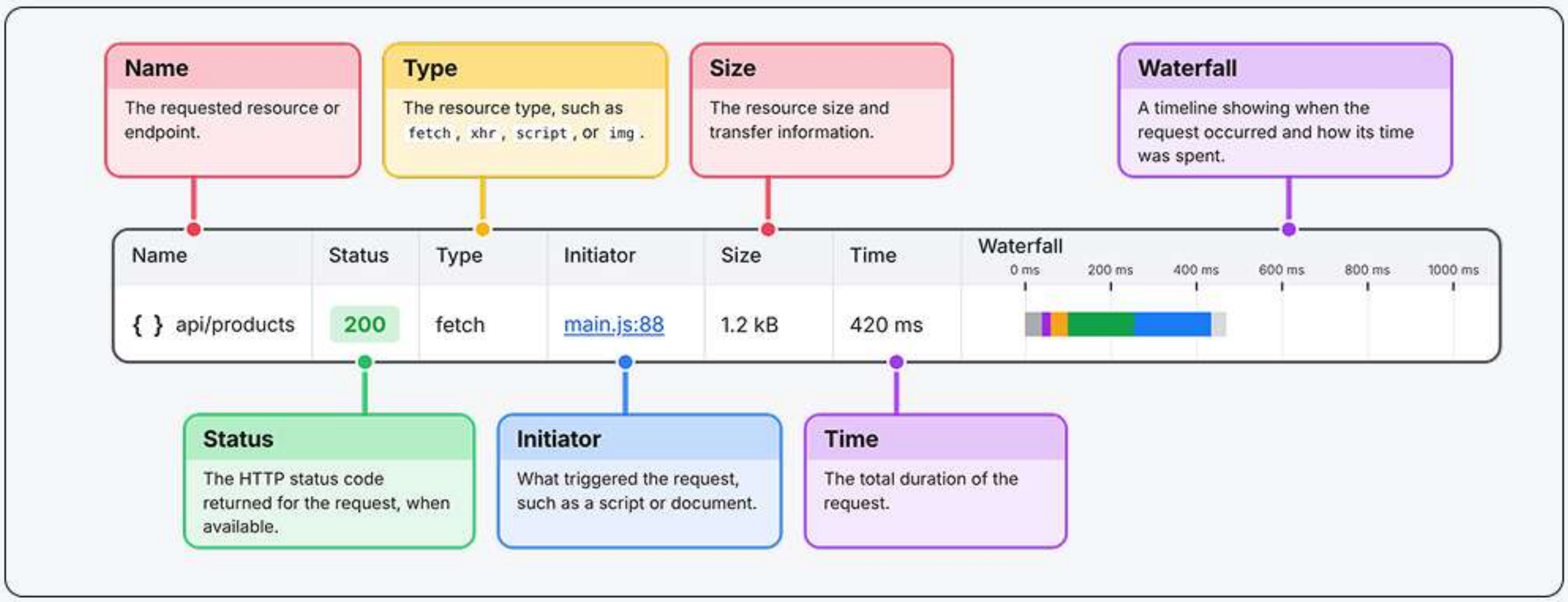
Whenever a page loads an image, fetches API data, downloads a script, or submits a form, you can inspect the corresponding network activity.



@this.girl.tech

How to Read a Request Row

Every row represents a network request or resource.



Name: The requested resource or endpoint.

Status: The HTTP response status code, when available.

Type: The resource category, such as fetch, xhr, script, or img.

Initiator: What triggered the request.

Size: The resource's size and transfer information.

Time: How long the request took.

Read the columns from left to right to understand each request.

Start with Name, Status, Type, and Time when investigating a problem.

@this.girl.tech

Understanding Status Codes

The status code tells you the outcome of a request.

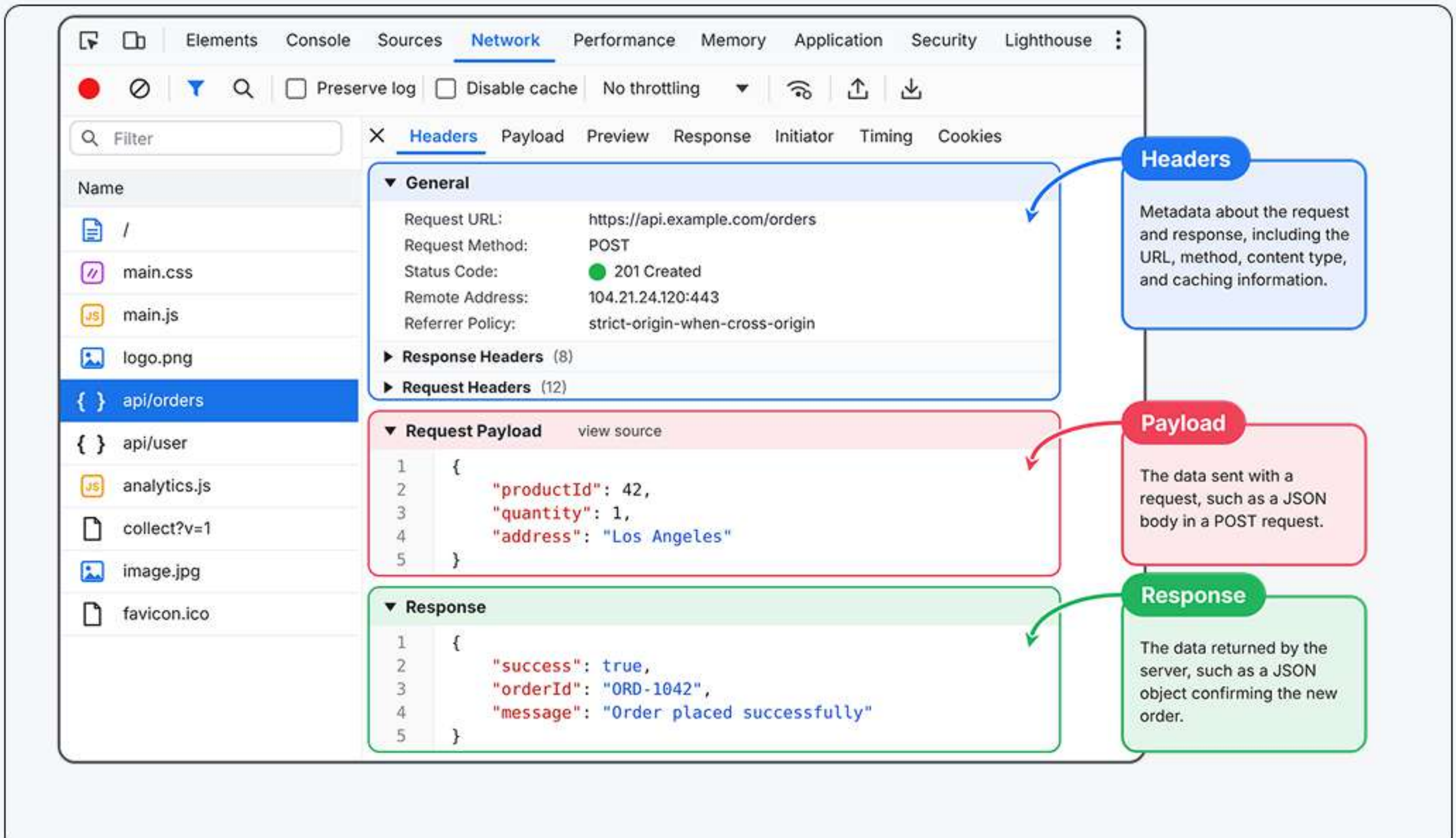
Code	Meaning	Recommended example
200	OK	Request succeeded
201	Created	Resource created
204	No Content	Request succeeded; no response body
301 / 302	Redirect	Resource redirected to another URL
400	Bad Request	Invalid request
401	Unauthorized	Authentication is required or invalid
403	Forbidden	Access is not permitted
404	Not Found	Resource not found
429	Too Many Requests	Rate limit exceeded
500	Internal Server Error	Server encountered an unexpected error
503	Service Unavailable	Service temporarily unavailable

A status code tells you what happened, but not always why.

@this.girl.tech

Headers vs. Payload vs. Response

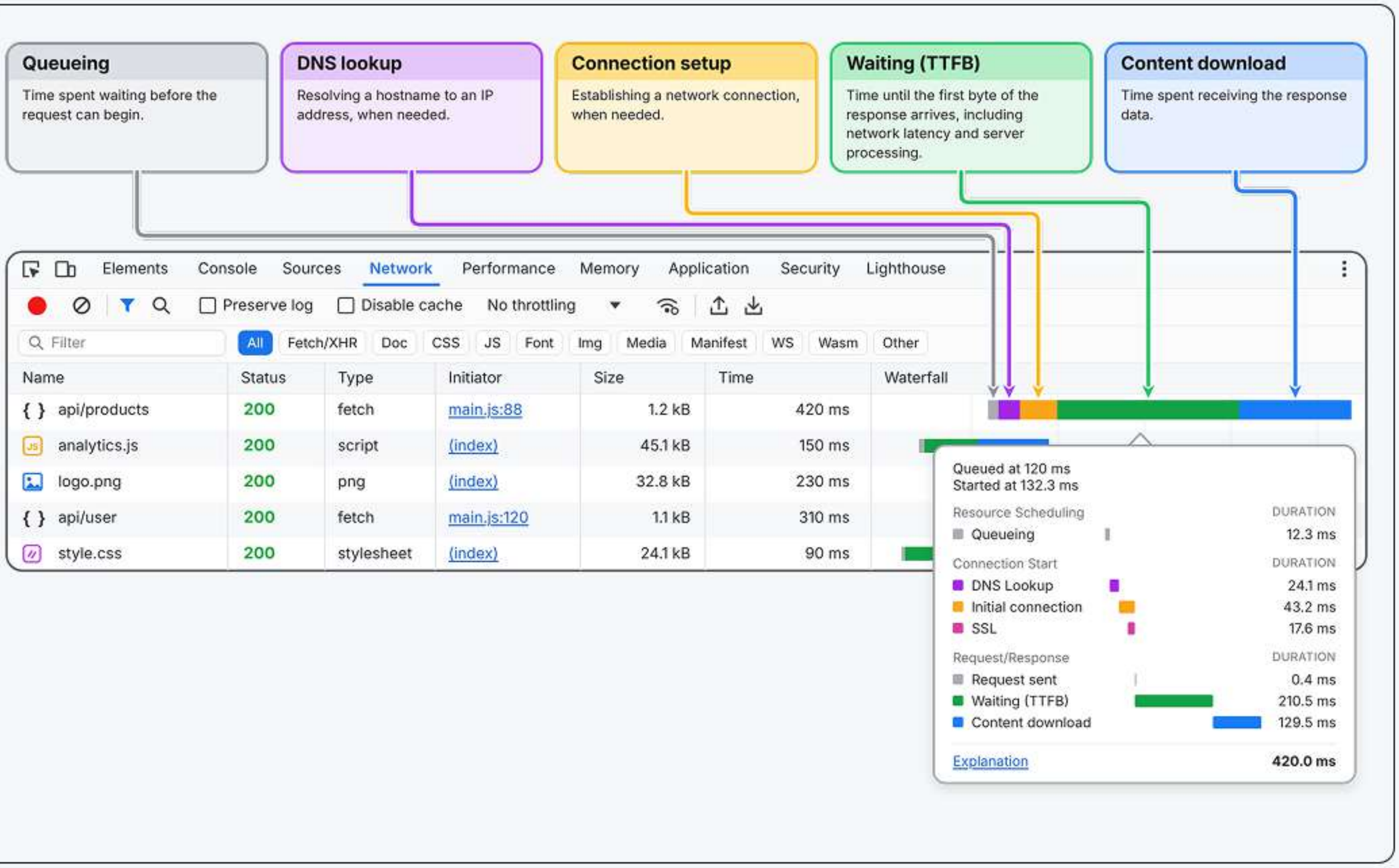
Click a request to inspect its details.



Inspect what you sent and what you received.

How to Read the Waterfall

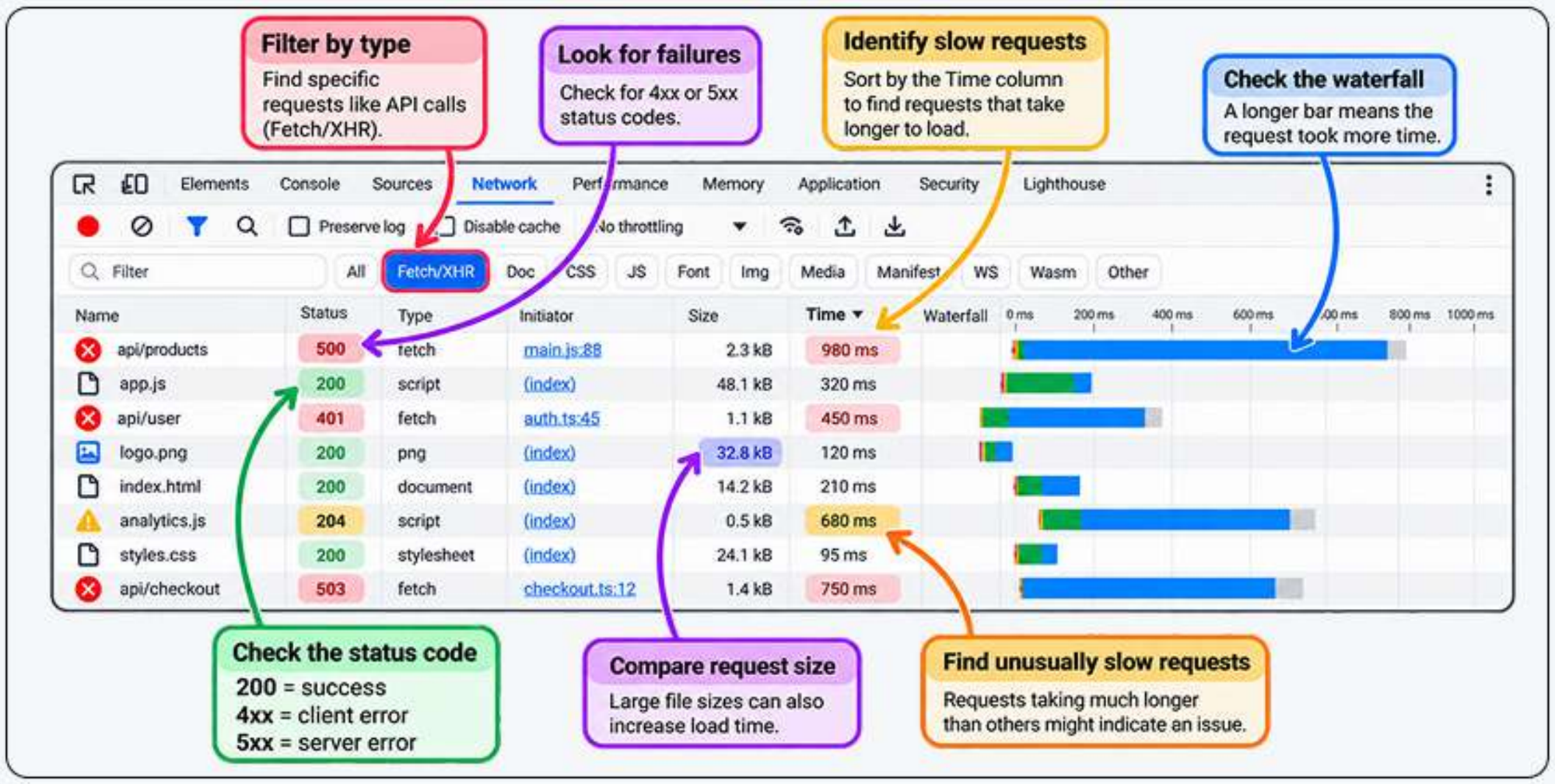
The Waterfall visualizes when requests happen and how long they take.



Not every request uses every phase.
Find out where the time went, not just how long the request took.

Finding Slow and Failed Requests

When a page feels slow or a feature stops working, start with the Network tab.



1. Filter by Fetch/XHR to focus on API requests.
2. Look for failed or unusually slow requests.
3. Check the status code.
4. Inspect the request URL, payload, and response.
5. Use the evidence to investigate the cause.

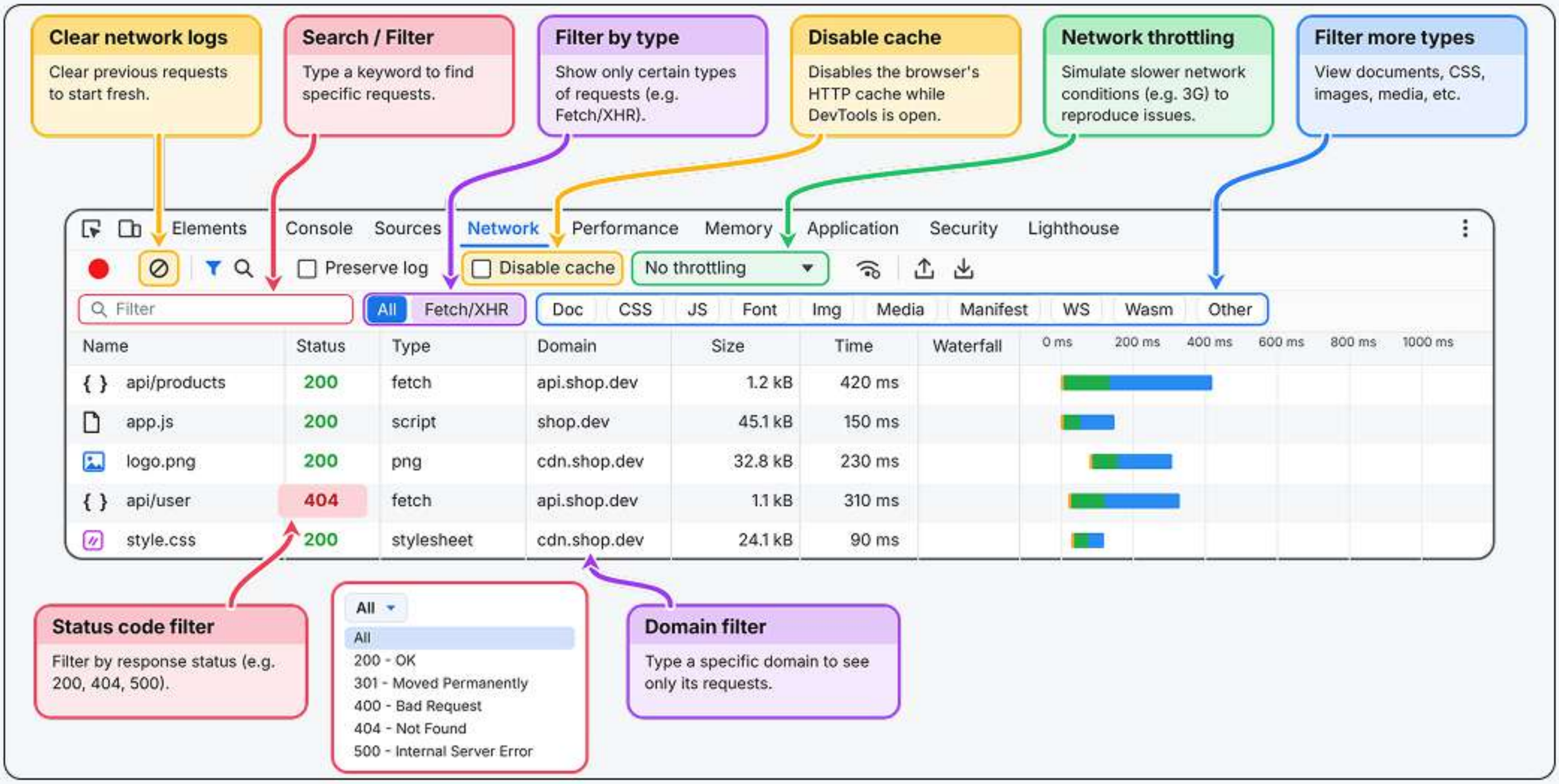
A failed request might result from an incorrect URL, authentication problems, server errors, network failures, or other issues.
A slow request might involve network latency, server processing, resource transfer, or time spent waiting before the response arrives.

Investigate before changing your code.

@this.girl.tech

Filters That Make Debugging Easier

The Network panel can contain hundreds of requests. Filters help you find the one you need.



Inspect what you sent and what you received.

A Real Debugging Example

Your products aren't loading. What do you check?

Imagine your frontend sends this request:

Name	Status	Type	Initiator	Size	Time	Waterfall
{ } api/products	500	fetch	main.js:88	1.2 kB	420 ms	0 ms · 200 ms · 400 ms · 600 ms · 800 ms · 1000 ms

Here's how to investigate:

1. Check whether the request URL is correct.
2. Inspect the response for an error message.
3. Check the status code.
4. Review the server logs.
5. Investigate the backend error.

What different results might suggest:

Status	Possible cause
404	Check the route or resource.
401	Check authentication (token, cookies, headers).
500	Investigate the server error and logs.
200 (empty data)	Check whether the response data matches what the UI expects.

These are starting points, not definitive diagnoses. Inspect the response and logs to narrow down the cause.

The same symptom can have different causes. Follow the evidence.

@this.girl.tech

THE NETWORK TAB MAKES THE INVISIBLE VISIBLE.

It helps you understand:

- **What the browser requests.**
- **What the server returns.**
- **Where requests fail.**
- **Why requests take time.**

The next time something breaks, don't guess.

Open DevTools. Inspect the request. Follow the evidence.

Save this for your next debugging session.

Comment “NETWORK” if you want more practical developer guides like this.

@this.girl.tech